植物园智能引导车

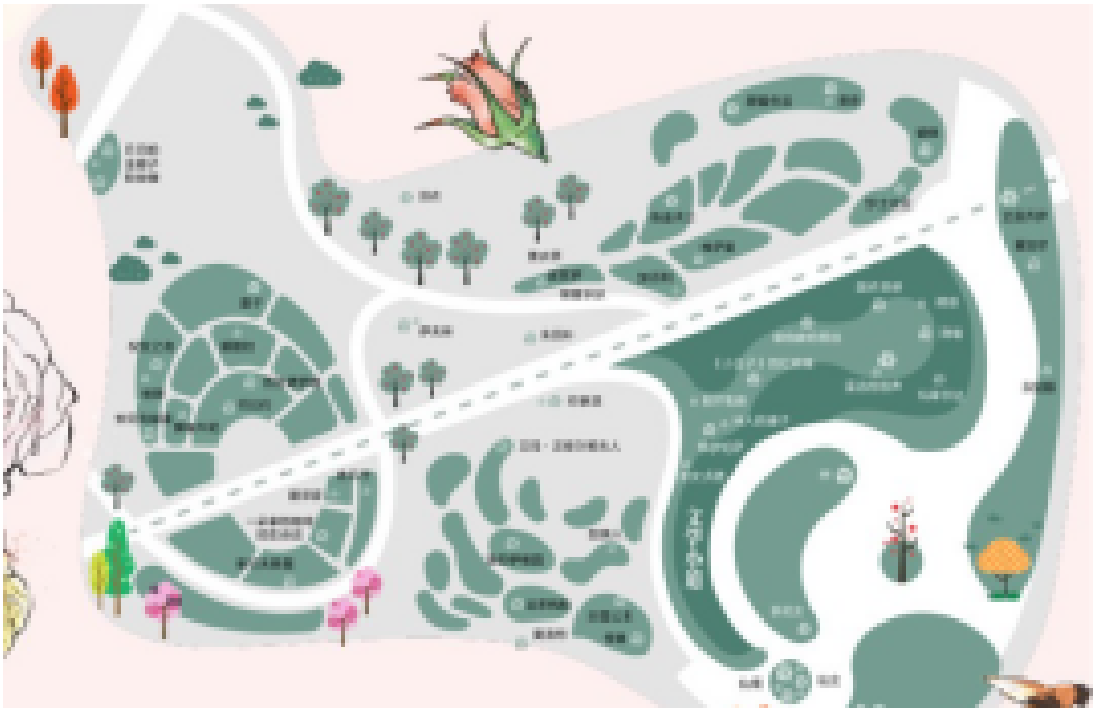
1. 应用场景分析

1.1 场景背景

- **用户痛点**：
  - 游客不熟悉园区布局，难以高效找到目标展区（如“热带雨林馆”）。
  - 传统导览器交互单一，缺乏实时性和趣味性。
- **解决方案**：
- 智能导览车提供**自主导航**+**语音交互**，结合LLM实现个性化讲解、对话。

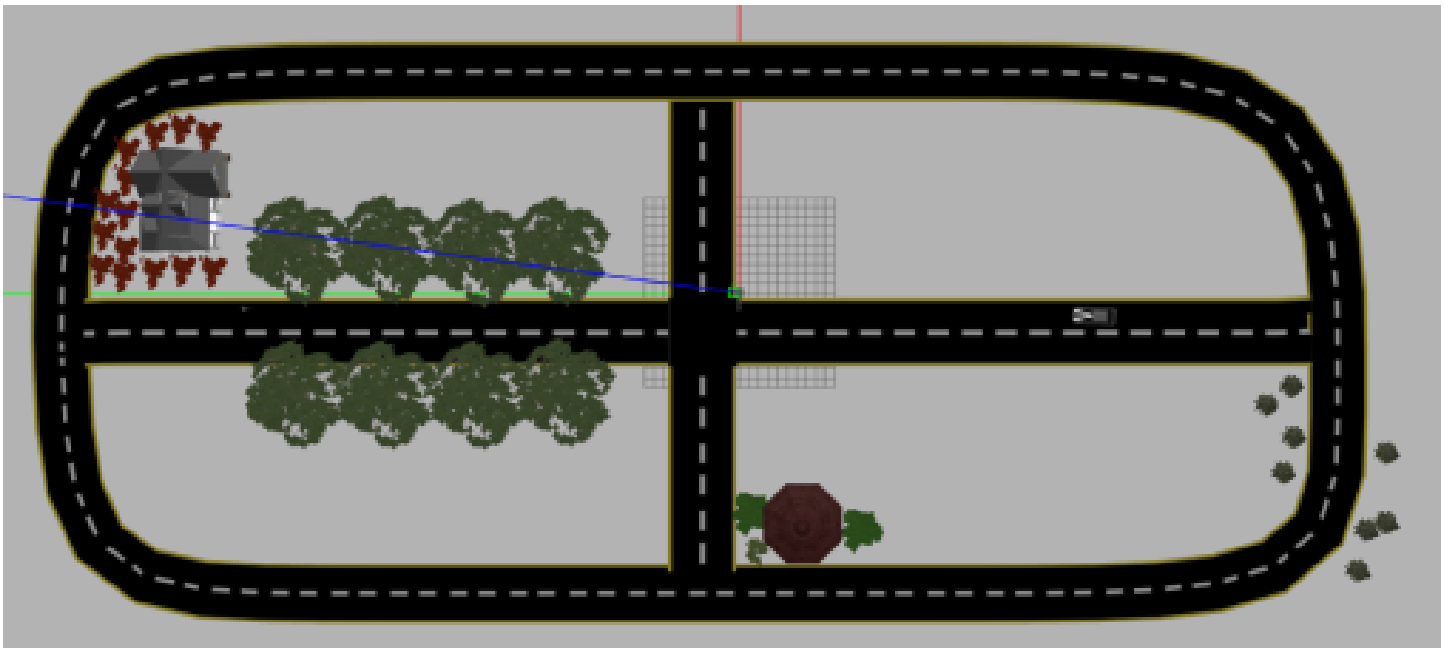
1.2 明确应用场景环境

物理环境：植物园内无车场景。

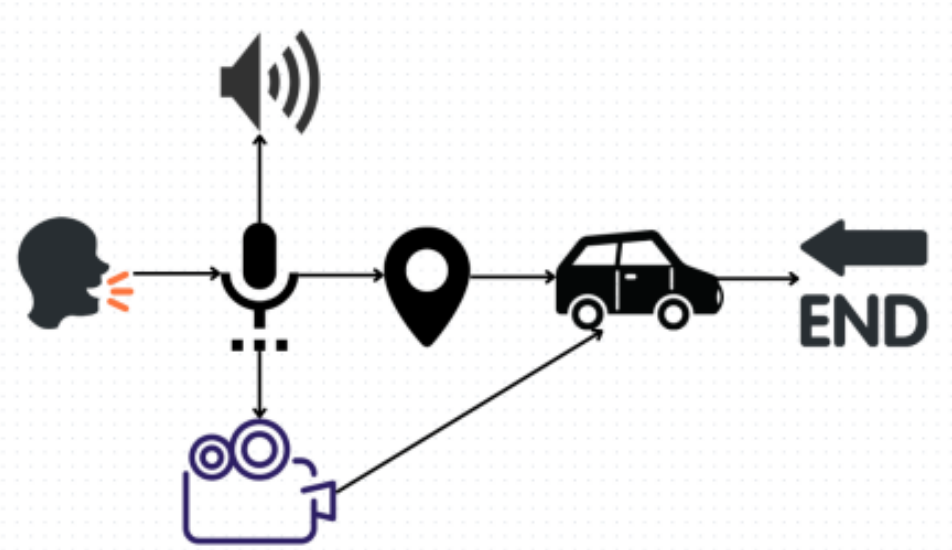


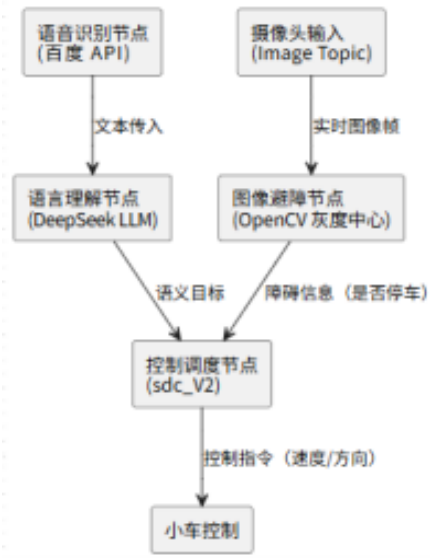
- **道路特征**：
  - 铺设车道或硬化路面，宽度2-3米（适配小型导览车）。
  - 车道线为彩色标识（如绿色生态主题），光照条件稳定（树荫区域需考虑阴影干扰）。
- **网络覆盖**：Wi-Fi 6/5G全覆盖，支持低延迟云通信（用于LLM交互）。

抽象后的场景粗糙建模设计：



1.3 任务定义





任务类型	功能描述
语音唤醒	游客通过指令（如“去玫瑰园”）唤醒导览车，支持多语言识别。
路径导航	基于A*算法规划最优路线，结合车道线跟踪（保留原车道识别模块）。
实时讲解	通过LLM生成展区科普内容（如植物习性），语音播报+屏幕图文展示。
安全避障	激光雷达检测游客突发闯入，触发低速绕行或停车（简化原动态避障逻辑）。

---

2. 基本方案：

2.1 方案概述

智能语音导览车将在植物园内服务，提供以下功能：

1. **语音导航服务**：游客通过自然语音指令（如"带我去玫瑰园"）唤醒导览车，自动规划最优路线并引导至目的地。
2. **实时科普讲解**：结合大语言模型（LLM），在行驶过程中为游客提供生动的植物科普讲解和趣味知识。
3. **多语言支持**：支持中英文等多种语言的语音交互，满足国际化游客需求。
4. **安全巡航**：通过激光雷达和视觉识别实现自主避障，确保在游客密集区域的安全行驶。
5. **智能交互界面**：配备触摸屏显示路线地图和景点信息，提供更直观的交互体验。

2.2 导航方式

机器人利用**路径规划、车道线识别**来导航，并结合自动避障以保证行驶安全：

- **路径识别：** *A（A-Star）启发式搜索算法*，结合二维网格地图，对植物园部的空间布局进行建模与导航。
- **车道线识别**：地面有明显的白色/黄色车道线，机器人可跟随车道线行驶。
- **自动避障**：通过超声波/激光雷达传感器，机器人可检测行人或障碍物，自动停下或绕行

2.3 硬件方案

组件	作用
高清摄像头	识别车道线、标识牌
超声波/雷达传感器	避障，检测行人等障碍物
电机驱动系统	机器人行驶
麦克风	定向麦克风阵列，提升语音交互体验。

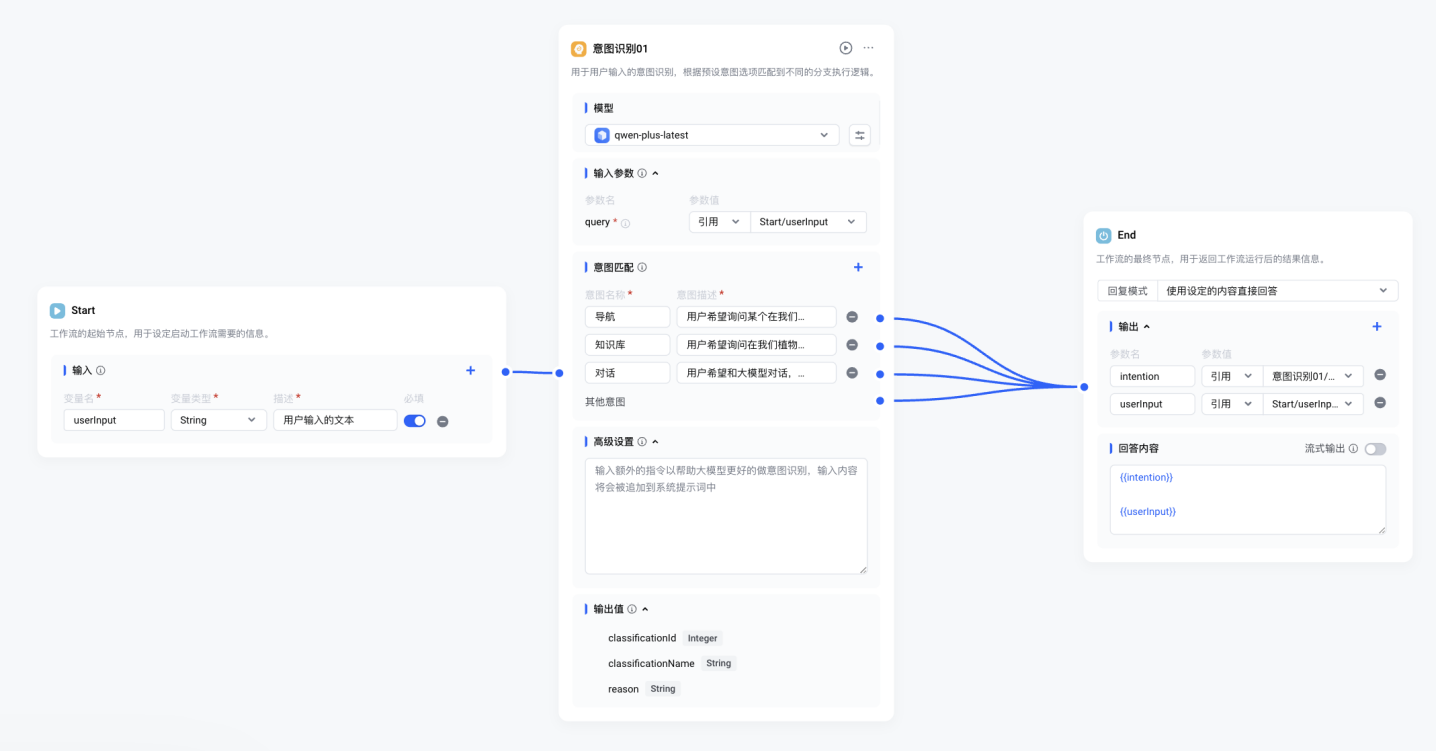
2.4 语音识别接入

语音交互模块采用百度语音识别 SDK，配合本地麦克风录音功能，实现语音到文本的转换。在识别出文本后，我们通过调用后序意图识别模型，对语义进行理解，提取出用户意图中涉及的话题、地点。整个过程仅需 1~2 秒，可实现较自然的人机交互体验。

2.5 LLM模型接入

2.5.1 意图识别

本部分调用大语言模型搭建工作流，对用户输入信息进行意图识别，工作流示意图如下：



最终导向四个分支——导航、知识库回答、LLM模型对话、无效输入。

2.5.2 LLM模型对话（趣味讲解）

通过接入DeepSeek API，利用特色定制的prompts，驱动大语言模型针对我们项目的场景为用户提供合适、有趣味的对话讲解内容。

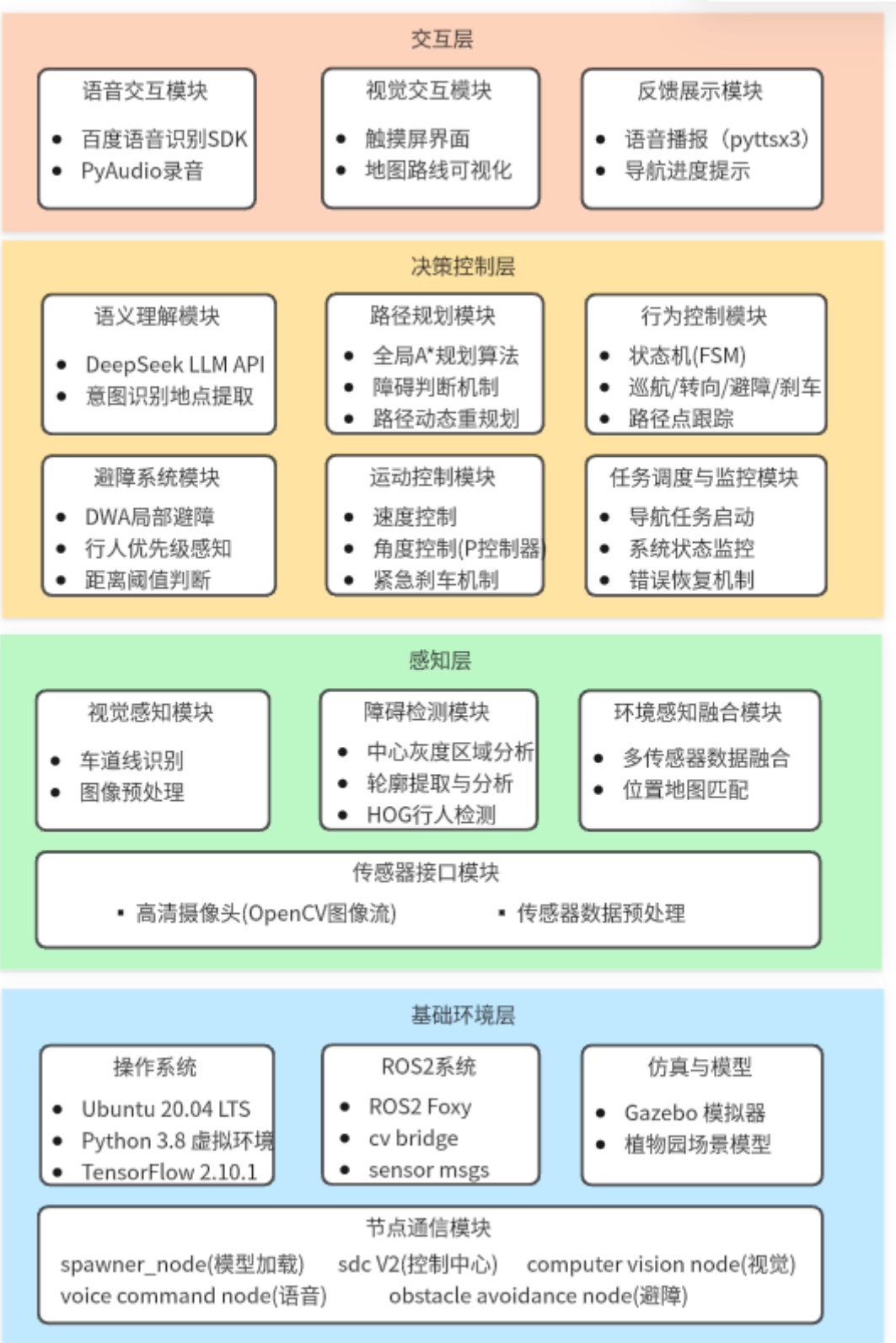
2.6 预期效果

- **游客体验**：找路时间减少60%，讲解互动性提升。
- **管理成本**：替代人工导览员，支持多语言覆盖。

---

3. 技术调研

3.1 系统架构



3.2 核心技术模块调研

3.2.1 车道线识别技术

当前车道识别技术多样，如传统图像处理、深度学习语义分割、目标检测+多传感器融合、Transformer+BEV感知等方式各有千秋。

需求维度	推荐方案	理由	缺点
实时性强	传统图像处理	快速部署，硬件要求低	易受反光、阴影、磨损车道线干扰。
高精度	深度学习语义分割	鲁棒性最优，适配复杂环境	需大规模标注数据集，以及计算资源需求较高
动态环境适应	目标检测+多传感器融合	平衡速度与抗干扰能力	硬件成本高，需解决多传感器时空同步问题。
长距离导航	Transformer+BEV感知	支持远距离车道线连续跟踪	模型参数量大，训练数据需求极高

为了确保安全、高速的车道识别过程，计算量小，实时性的基于传统计算机视觉的方法，更适合我们当前的模拟车道线识别技术的选型。该方式采用模块化设计，将车道线检测任务分解为分割、估计、清理和信息提取四个主要阶段，每个阶段都针对特定的技术挑战提供了相应的解决方案。

3.2.2 动态避障系统

算法类型	原理	优点	缺点
动态窗口法（DWA）	在速度空间采样可行轨迹，结合碰撞检测选择最优路径	实时性强（<50ms），适合动态环境	全局路径优化能力弱
人工势场法（APF）	障碍物产生斥力，目标点产生引力，引导机器人沿合力方向移动	逻辑简单，易于实现	易陷入局部最优，震荡现象
强化学习（RL）	通过训练模型学习避障策略，适应复杂未知环境	自适应性强，支持端到端决策	需大量训练数据，实时推理依赖算力
混合算法	全局A*规划 + 局部DWA避障，结合语义分割过滤动态障碍	平衡全局最优与实时避障	系统复杂度高

适宜方案

- **核心逻辑**：全局路径由A*算法生成，局部避障采用DWA动态调整。
- **优化策略**：
  a. **动态障碍物分类**：通过视觉区分行人（高优先级）与静态物体（低优先级）。
  b. **速度自适应**：根据障碍物距离调整巡航速度（如距离<2m时降速至0.5m/s）。

c. **安全阈值**：设定最小安全距离（0.3m），触发紧急刹车。

3.2.3 路径规划

在本系统中，路径规划模块负责计算智能引导车从当前位置出发到达目标位置之间的最优路径。为了实现高效且稳定的路径搜索，我们采用经典的 *A（A-Star）启发式搜索算法*，结合二维网格地图，对植物园内部的空间布局进行建模与导航。

（1）地图构建

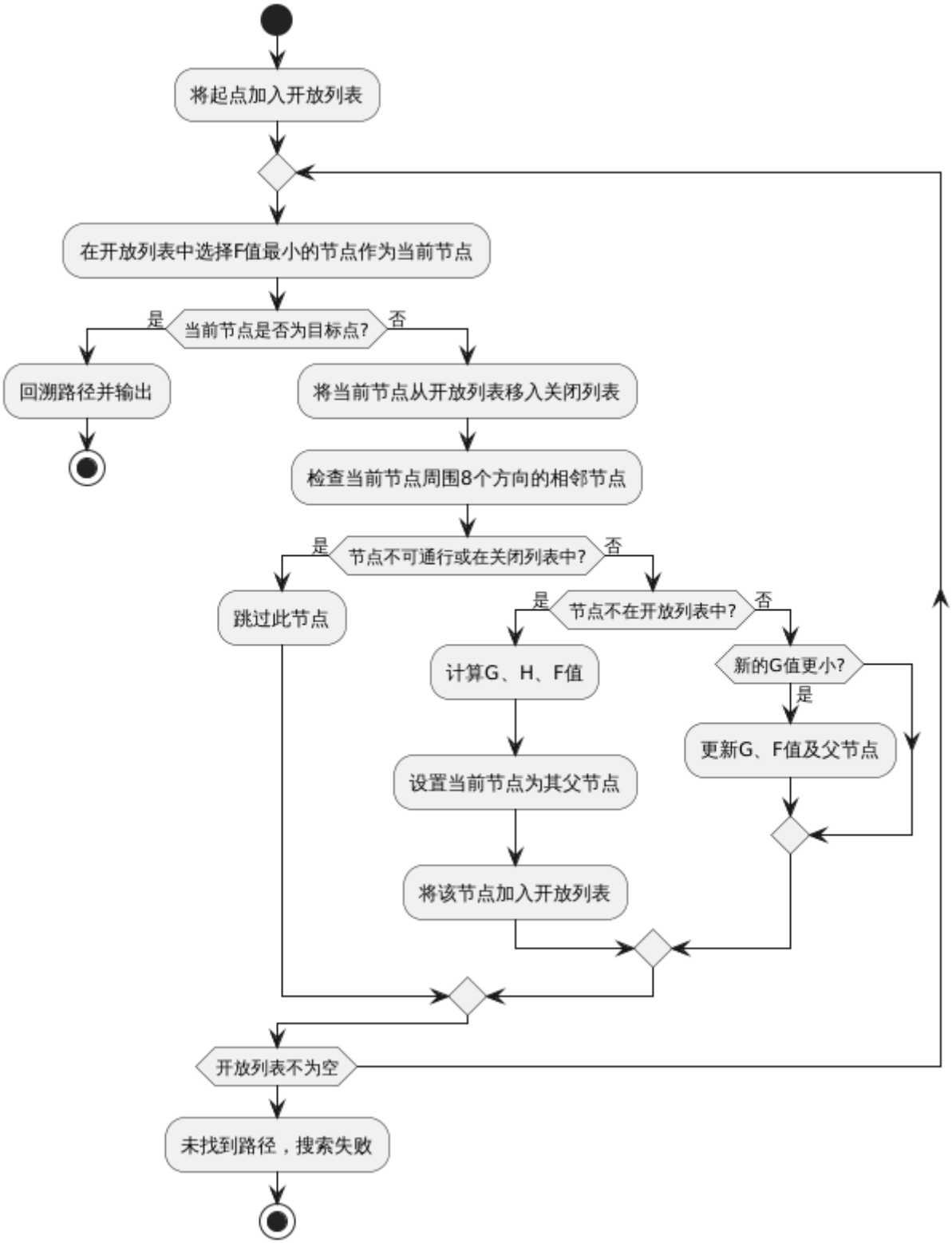
系统将整个植物园划分为一个二维矩阵，每一个矩阵单元代表一个位置点，称为“节点”。每个节点包含如下信息：

- 坐标位置；
- 是否为障碍；
- 启发式评估代价（H值）；
- 起点到当前节点的实际代价（G值）；
- 综合代价（F = G + H）；
- 父节点（用于回溯路径）。

（2）A*算法简介

A*算法是一种启发式路径搜索算法，结合了Dijkstra算法的最短路径思想与启发函数预测目标方向的能力。具体流程如下：

1. 将起点加入开放列表；
2. 在开放列表中选择F值最小的节点作为当前节点；
3. 判断是否到达目标点，若是则回溯路径；
4. 若否，将当前节点移动到关闭列表，对其周围8个方向的相邻节点进行如下操作：
    - 若节点不可通行或已在关闭列表中，跳过；
    - 若不在开放列表中，计算其G、H、F值，并设置当前节点为其父节点；
    - 若已在开放列表中，比较新的G值与原G值，若更小则更新其父节点和代价；
5. 重复步骤2至4，直到找到路径或开放列表为空（即无解）。



（3）系统实现与优化

在实际代码实现中，我们通过以下方式提升了算法效率与适应性：

- **八方向搜索**：采用8邻域搜索方式（上下左右+四个对角线方向），提升路径自然性与连贯性；
- **障碍判断机制**：地图节点通过 `obstacle` 标志位标记障碍区域，避免引导车碰撞；
- **路径可视化**：规划结果以矩阵方式实时输出，通过串口或显示模块反馈路径，便于调试与展示；
- **地图边界处理**：增加对越界访问的判断，保证算法鲁棒性；
- **起点自动更新**：支持路径动态重规划，在导航中若遇障碍可重新计算路径。

（4）应用场景

路径规划模块在整个系统中具有核心地位，典型应用包括：

- 从入口至目标园区的导航；
- 从园区至卫生间、游客中心等特殊位置的路径引导；
- 在遇到临时障碍（如封路或人流量过大拥挤或临时闭馆）时自动重新规划路径。

3.3 真实场景中需要的通信与网络架构

3.3.1 车-云通信方案（实际开发过程中使用简易模拟）

- **协议选择**
  - MQTT协议（轻量级IoT协议）
  - 5G URLLC（超可靠低时延通信）
- **边缘计算节点**
  - 部署在园区各设施处的边缘服务器
  - 实现本地化数据处理（延迟<50ms）

3.4 关键技术验证计划

3.4.1 仿真测试

- **工具链**
  - ROS2节点架构设计
  - 仿真平台构建虚拟植物园
  - Gazebo进行动力学仿真

---

4. 技术路线：

4.1 系统平台构建

该项目基于 Ubuntu 20.04 LTS 环境，搭建 ROS2 Foxy 框架作为系统通信中枢。配套集成 Gazebo 仿真平台，用于构建真实比例的植物园模拟环境，支持传感器数据、物理仿真与路径可视化。

为实现模块化开发与部署，系统采用 Python3 虚拟环境，配置图像处理与神经网络相关依赖，包括 OpenCV、TensorFlow、GStreamer 等，确保仿真与实际部署的统一性与可移植性。

4.2 感知模块

感知模块采集车辆周边环境信息，完成对道路、障碍物、目的地、道路标识的识别与分析，OpenCV提供轻量级图像处理能力，适用于实时运行场景；雷达/超声波模拟为物理感知提供补充。

感知任务	所用技术	说明
车道线识别	OpenCV（Canny边缘检测 + 霍夫直线变换）	用于识别地面黄/白色导向线，引导车辆沿路径前进，简洁高效，适配植物园直线/弯道环境
障碍物检测	图像阈值法、轮廓检测、超声波/激光模拟	识别车前障碍物（如人、车），实现停车/避让功能，算法轻量，适合嵌入式部署
图像处理框架	GStreamer（OpenCV视频流支持）	支持 AVI 视频输入输出及调试录像，可视化过程

4.3 路径与行为规划

系统支持基于目标点的路径自主规划功能，系统将自动在已知地图中生成全局最优路径。路径规划主要分为两个阶段：

- **全局路径规划**：根据植物园拓扑结构，生成从当前位置到目标点的路径点序列；
- **局部运动规划**：在执行路径跟踪过程中，动态结合车道线引导与障碍物信息，实时判断车辆转向、变速、停车等行为。

植物园为静态已知环境，路径与行为规划模块在感知数据基础上，制定行为策略与路径，引导车辆完成任务。

决策任务	所用技术	说明
路径规划	预设地图、路径点规划算法	系统根据目标点计算从当前位置到目标路径，适用于植物园已知拓扑结构的规划需求
行为判断	状态机	根据感知结果实时切换行为状态（如避障、跟线、掉头）
局部运动策略	路径点跟踪 + 车道偏移判断	保持在车道中心行驶，结合转角调整、避障绕行等策略
目标接收机制	云台模拟下发目标：目的地坐标	模拟智能云平台任务调度机制，实现车辆“任务驱动”导航

4.4 控制模块

控制模块将高层决策结果转化为车辆执行动作，完成速度与方向控制，本项目用gazebo模拟。

控制任务	所用技术	说明
转向控制	转角误差计算、比例控制	根据车头偏离车道中心的角度调整转向角，控制舵机方向；P控制响应快、实现简单
速度控制	状态机输出、常量速度、刹车命令	在直行、避障、停止等状态下控制速度指令，适配 Gazebo 模拟底盘
运动执行	Gazebo 控制接口	将控制信号作用于车辆模型，实现真实转弯/加减速动作

---

5. 机器人底座：

本项目基于 Gazebo 仿真平台构建了具备自主导航能力的引导机器人模型，其底座结构设计包括传感器输入、控制执行与物理反馈等功能模块，并通过 ROS2 框架实现模块间数据交互与行为协调。机器人底座主要模拟真实车辆在植物园内行驶的场景，底层功能划分如下：

5.1 物理模型

- **底盘模型**：使用 `prius_hybrid` 模拟车辆，具备四轮结构、车体惯性属性和转向反馈系统，已集成于 Gazebo 模型库。
- **运动控制接口**：通过 ROS2 中 `joint_velocity_controller` 控制节点实现车速与方向调节，支持匀速直行、变道、转向等操作。
- **动力控制方式**：车辆采用比例控制（P-Control）调整前轮转向角，模拟实际车道跟随过程。

5.2 传感器模拟与感知输入

传感器类型	模拟方式	功能
摄像头（前视）	Gazebo 模拟相机 → OpenCV 图像流	实现车道线识别、障碍物检测、目标感知等任务。
目标检测/视觉感知	OpenCV 、图像处理算法	提取车道线、区域障碍、标志信息
路径信息输入	鼠标点击 SatView 设置目标点	用于导航路径规划

5.3 控制执行模块

控制模块主要基于状态机逻辑和 ROS2 通信结构实现。

5.3.1 行为控制

根据路径规划与视觉结果输出控制量。

5.3.2 控制逻辑

采用 FSM（有限状态机）结构判断行为状态（直行、转弯、避障、刹车），并输出常量速度与目标角度。

状态名	判定条件	行为输出
巡航模式	正常路径内，无障碍物	匀速前进 + 微调转角
转向模式	检测到道路曲率增加或到达路径弯点	动态调节角度，速度适度减小
避障模式	前方检测到障碍（人/车/物体）	停止或规划绕行路线
刹车模式	到达目标点或发生紧急情况	立即速度设为 0，执行刹车动作

5.3.3 控制策略：

- **速度控制**：恒定巡航速度，在避障或转弯状态下动态调整；
  - 默认使用 **常量巡航速度**（如 1.0 m/s），保证仿真稳定。
  - 在转向或避障状态下，**根据偏角和距离动态调整速度**，以防止急弯倾斜或碰撞。
  - 实现机制为设定 **target_speed**，由状态机动态修改并发布。
- **角度控制**：基于车道线偏差进行比例转向控制，保持在路径中心；
  - 使用 **车道线中线偏差** 或 **路径点朝向误差**计算转向角。
  - 基于比例控制（P控制）算法，将偏差放大为可执行的角度命令
- **刹车机制**：检测前方障碍立即将速度设为 0，实现刹车响应。

---

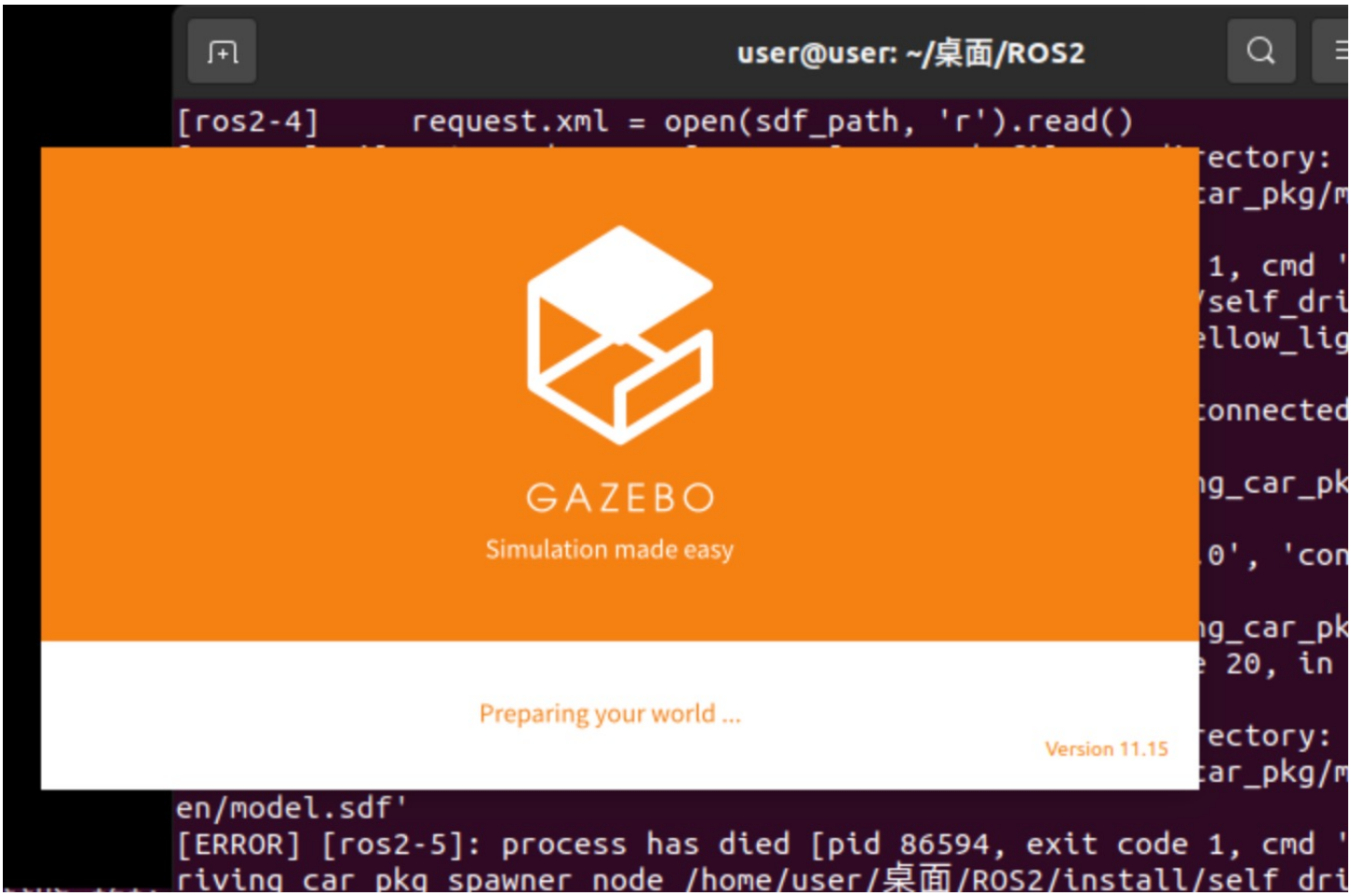
6. 环境和数据资源准备：

6.1 系统基础环境配置

- **操作系统**：Ubuntu 20.04 LTS。
- **ROS2 版本**：Foxy Fitzroy。
- **Gazebo 模拟器**：

代码块

```
sudo apt update
sudo apt install -y ros-foxy-gazebo-ros-pkgs
```

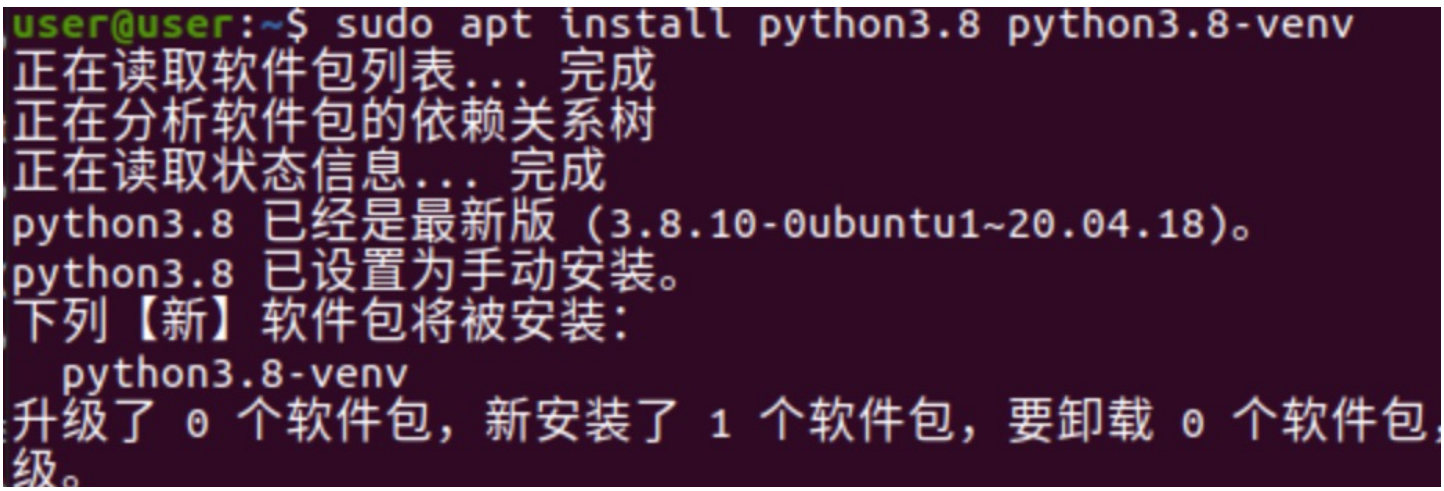


6.2 Python 虚拟环境配置

- 安装 Python3.8 及虚拟环境工具

代码块

```
venv
sudo apt install python3.8 python3.8-venv
```



- 创建并激活项目虚拟环境：

代码块

```
python3.8 -m venv tf24_env
source tf24_env/bin/activate
```

```
user@user:~$ python3.8 -m venv tf24_env
user@user:~$ source tf24_env/bin/activate
```

6.3 项目依赖安装与说明

- **OpenCV 3.4.15（contrib 版）**

代码块

```
pip install opencv-contrib-python==3.4.15.55
```

```
(tf24_env) user@user:~$ pip install opencv-contrib-python==3.4.15.55
Collecting opencv-contrib-python==3.4.15.55
  Downloading opencv_contrib_python-3.4.15.55-cp38-cp38-manylinux2014_aa
l (37.2 MB)
     |[                                    ]| 37.2 MB 2.8 MB/s
Requirement already satisfied: numpy>=1.19.3 in ./tf24_env/lib/python3.8
ckages (from opencv-contrib-python==3.4.15.55) (1.24.4)
Installing collected packages: opencv-contrib-python
Successfully installed opencv-contrib-python-3.4.15.55
```

- **TensorFlow 2.10.1**

代码块

```
pip install tensorflow==2.10.1
```

```
(tf24_env) user@user:~$ pip install tensorflow==2.10.1
Collecting tensorflow==2.10.1
  Using cached tensorflow-2.10.1-cp38-cp38-manylinux_2_17_a
aarch64.whl (1.9 kB)
Collecting tensorflow-cpu-aws==2.10.1; platform_system == "
machine == "arm64" or platform_machine == "aarch64")
  Downloading tensorflow_cpu_aws-2.10.1-cp38-cp38-manylinux
ux2014_aarch64.whl (213.7 MB)
     |                                                | 213.7 MB 126 kB/s
Requirement already satisfied: gast<=0.4.0,>=0.2.1 in ./tf2
ite-packages (from tensorflow-cpu-aws==2.10.1; platform_sys
latform_machine == "arm64" or platform_machine == "aarch64"
) (0.4.0)
Collecting opt-einsum>=2.3.2
  Using cached opt_einsum-3.4.0-py3-none-any.whl (71 kB)
```

- **Matplotlib、VisualKeras**pip

代码块

```
pip install matplotlib visualkeras
```

```
(tf24_env) user@user:~$ pip install matplotlib
Collecting matplotlib
  Downloading matplotlib-3.7.5-cp38-cp38-manylinux_2_17_aarc
rch64.whl (11.4 MB)
     |                                                | 11.4 MB 3.5 MB/s
Collecting pyparsing>=2.3.1
  Downloading pyparsing-3.1.4-py3-none-any.whl (104 kB)
     |                                                | 104 kB 55.5 MB/s
Collecting cycler>=0.10
  Downloading cycler-0.12.1-py3-none-any.whl (8.3 kB)
Collecting contourpy>=1.0.1
  Downloading contourpy-1.1.1-cp38-cp38-manylinux_2_17_aarch
ch64.whl (285 kB)
     |                                                | 285 kB 8.2 MB/s
Requirement already satisfied: packaging>=20.0 in ./tf24_env
```

```
(tf24_env) user@user:~$ pip install visualkeras
Collecting visualkeras
  Downloading visualkeras-0.1.4-py3-none-any.whl (17 kB)
Requirement already satisfied: pillow>=6.2.0 in ./tf24_en
ckages (from visualkeras) (10.4.0)
Collecting aggdraw>=1.3.11
  Downloading aggdraw-1.3.16-cp38-cp38-manylinux_2_17_aar
h64.whl (970 kB)
     |                                                    | 970 kB 282 kB/s
Requirement already satisfied: numpy>=1.18.1 in ./tf24_en
ckages (from visualkeras) (1.24.4)
Installing collected packages: aggdraw, visualkeras
Successfully installed aggdraw-1.3.16 visualkeras-0.1.4
```

- **Pygame**

代码块

```
pip install pygame
```

```
(tf24_env) user@user:~$ pip install pygame
Collecting pygame
  Downloading pygame-2.6.1-cp38-cp38-manylinux_2_17_aarch
4.whl (13.6 MB)
     |                                                    | 13.6 MB 1.1 MB/s
Installing collected packages: pygame
Successfully installed pygame-2.6.1
```

- **GStreamer** 为 OpenCV 提供强大的视频输入输出接口，支持 .avi 等格式保存训练或推理过程中的视频帧。

6.4 ROS2 项目激活与启动

- 构建工程并生成 install 工作空间：

代码块

```
source tf24_env/bin/activate
source /opt/ros/foxy/setup.bash
cd ROS2-Self-Driving-Car-AI-using-OpenCV
colcon build
source install/setup.bash
```

```
(tf24_env) user@user:~/桌面/ROS2$ source /opt/ros/foxy/setup.bash
(tf24_env) user@user:~/桌面/ROS2$ source install/setup.bash
(tf24_env) user@user:~/桌面/ROS2$ ros2 launch self_driving_car_pkg world_gazebo.launch.py
```

- 启动仿真世界与自动驾驶节点以进行仿真：

代码块

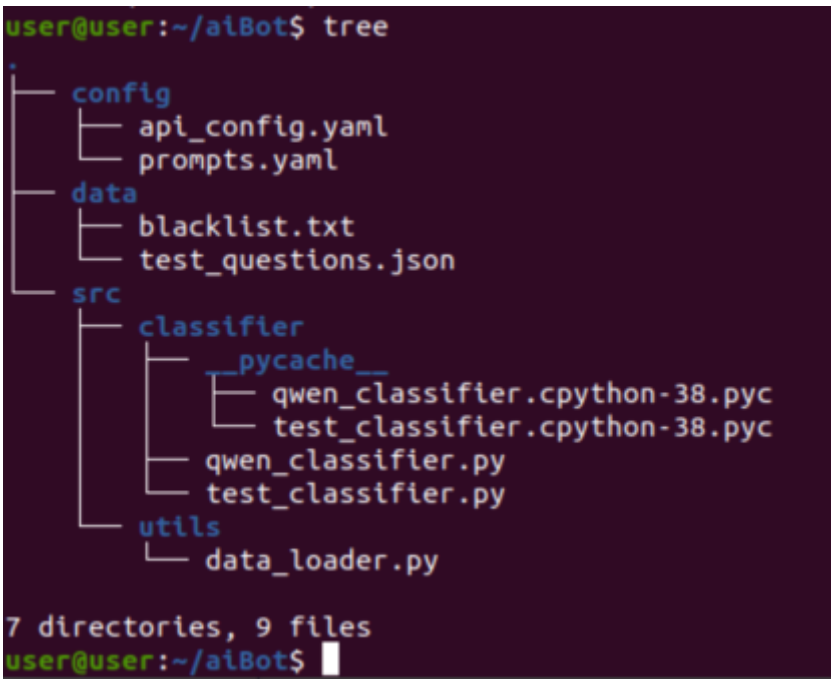
```
ros2 launch self_driving_car_pkg world_gazebo.launch.py
ros2 run self_driving_car_pkg computer_vision_node
```

6.5 数据资源准备

6.5.1 路径测试资源

我们准备了一些路径测试的资源，设置起点、终点、转弯、障碍等行为，测试当前路径规划与避障功能的效果

6.5.2 意图识别模型优化



在此测试项目中通过 `test_classifier` 调用 `test_questions` 不断测试 `qwen_classifier` 的意图识别准确率，不断优化 `prompts` ，提高命中率与准确率。

6.5.3 避障功能训练资源

参考链接如下：

News – Cityscapes Dataset

使用 `gtFine` 标注 `+ leftImg8bi` t图像，使用官方 `cityscapesscripts` 工具转为 `COCO` / `YOLO` 格式。选择 `YOLOv8` 等轻量模型。训练完成后模型可嵌入 `ROS2` 节点，实时图像识

别控制 `/cmd_vel`

---

7. 未来集成与优化

场馆状态监测 —— 结合植物园内智能信息系统，实现更精准的场馆人流量以及道路拥挤程度等信息的传递。

更多交互方式 —— 在未来现实中实现可以通过app或触摸屏、语音助手等呼叫该机器。

AI 语音助手 —— 增加多语言支持，适用于国际旅客。

机器人队列协作 —— 多台机器人互相通信，优化实时道路信息通知。